Efficient Gait Classification Using Skeleton and Pressure Data: Deep and Spiking Neural Network Approaches

Eduardo David Lotto, Domenico Plantamura

***Abstract*—Gait analysis is an important tool in the medical field, as identifying pathological gait patterns can aid in the diagnosis of a wide range of diseases. Several machine learning-based approaches have been proposed to automate gait classification using different types of sensors. In this paper, we present deep learning models that leverage 3D skeleton data and foot pressure measurements related to one normal and five pathological gaits. We introduce two single-modal models: a Gated Recurrent Unit (GRU)-based encoder for the skeleton data and a Convolutional Neural Network (CNN)-based encoder for the foot pressure data. Features extracted from each modality are then combined using a multi-modal fusion strategy to form a hybrid model. The pressure-based and skeleton-based models achieved classification accuracies of 48.9% and 94.2%, respectively. The hybrid model performed slightly better (94.8%) but incurs higher spatial and temporal costs.**

To improve efficiency and reduce latency, we also developed a Spiking Neural Network (SNN) based on 3D skeleton data, which proved to be the most effective modality for a spiking-based implementation. This lightweight model achieves acceptable performance (83.2% accuracy) and provides a promising foundation for future development.

When considering performance alone, the hybrid model proves to be the most effective, as it integrates information from both skeleton and foot pressure data. However, when evaluating the trade-off between performance and efficiency, the skeleton-based modality emerges as the most balanced choice.

***Index Terms*—Skeleton, Foot, Multi-modal, Spiking, Gait, Neural Networks**

I. Introduction

Gait analysis, defined as *the evaluation of the manner or style of walking, usually done by observing the individual walking naturally in a straight line* [1], is a valuable resource in the medical field. Detecting abnormal patterns can be useful in the diagnosis of a broad spectrum of diseases, from musculoskeletal disorders to neurological disorders. Gait data can be collected using various sensors to calculate parameters of interest that can be used to classify pathological patterns. Classification can be performed by hand, but requires complex steps: hence, machine learning-based approaches can be highly beneficial.

Two types of sensors can be employed for data collection: wearable and non-wearable. Wearable ones (such as inertial sensors attached to hips, knees, and ankles) guarantee higher accuracy, but they require complicated systems and can be uncomfortable. Due to these issues, it is easier to collect real-life gait data with non-wearable sensors [2]. This is the case of the dataset we used, which was acquired with Azure Kinect for 3D skeleton data and GW1100 for foot pressure measurements.

Several machine learning-based approaches for gait classification can be found in the literature. Some studies ([3], [4], [5], [6]) focus solely on 3D skeleton data, overlooking the fact that certain pathological gaits exhibit spatiotemporal similarities, which makes it challenging to distinguish them using only vision-based information. Other researches combine both skeleton and foot pressure data. However, they involve computationally intensive architectures such as DenseNet201 ([2]) or complex graph convolutional networks ([7]).

The purpose of our study is to classify six types of gait, one normal and five pathological (antalgic, lurching, steppage, stiff-legged, and Trendelenburg), using both 3D skeleton data and foot pressure data. To this end, we evaluated single-modal models that rely solely on either skeleton or pressure data, as well as a hybrid model that involves a multi-modal feature-level fusion, following the approach in [2].

We aim to strike a balance between classification accuracy and computational efficiency. To this end, we implemented a lightweight CNN-based encoder to extract features from foot pressure data, in contrast to previous studies that typically rely on significantly larger models. Furthermore, trying to improve efficiency and reduce latency, we created a baseline Spiking Neural Network based on 3D skeleton data, as it appeared to be the most effective modality for a spiking-based implementation and gait classification performance.

Finally, we compared all models by evaluating not only their accuracy, but also their simplicity and inference latency. The complete list of models is as follows:

- GRU-based model fed with skeleton data (SkGRU)
- CNN-based model fed with foot pressure data (PrCNN)
- Hybrid model (HyM) combining features extracted from SkGRU and PrCNN encoders
- Spiking Neural Network fed with skeleton data (SkSNN)

This report is structured as follows. In Sec. II we briefly review related studies and highlight their contribution to this work. Sec. III provides a high-level overview of the experimental setup. A detailed description of the dataset and preprocessing techniques is presented in Sec. IV. Sec. V includes a description of the models architectures and training setup. In Sec. VI the models are evaluated and compared in terms of performance, size and latency, while final considerations and concluding remarks are discussed in Sec. VII.

II. Related Work

Various works performed gait classification using different approaches. In particular, the work of Jun et al. [2] served as a fundamental basis for the development of our study. We used the dataset they collected with non-wearable sensors, followed their approach involving both single and multi-modal inputs, and reimplemented two of their proposed architectures. Following [4], where a GRU and a LSTM-autoencoder are proposed for extracting skeleton features, we chose to perform feature extraction rather than using the raw input data for gait classification. [2] and [7] inspired us to combine skeleton and foot pressure measurements: we developed a hybrid classification model based on multi-modal feature-level fusion. While Naseem et al. [7] and [2] adopted complex architectures such as a graph convolutional network and DenseNet201 respectively for foot pressure data analysis, we tried to take the opposite direction, designing a lightweight and simpler CNN architecture.

The normalization method proposed by Baptista et al. [8], which involved translating the origin to the hip center, followed by rotation and scaling, inspired us to adopt a similar strategy based on pelvis centering and hip-width scaling. Following the approach in [2] and in [5], we performed data augmentation by flipping the skeleton and foot pressure training data along the left-right axis. Inspired by [7], we also performed a 30° rotation on foot pressure data.

For what concerns 3D skeleton data, taking a reduced number of joints lightens the computation and can even improve the performances, as shown in [3], [6] and [2]. For this reason, we limited the analysis to two specific subsets of skeleton joints rather than using the full set: the first one excludes the joints of face and arms while the second one only includes the legs ones. The documentation of the Azure Kinect joints can be found in [9], from which we obtained the coordinate system, the hierarchy structure, and the complete list of joints. We considered a temporal window of 100 time steps as in [2].

We used [10] as a foundation for understanding and implementing Spiking Neural Networks, as it provides valuable insights into their underlying principles, training challenges, and potential benefits. Tao et al. [11] presented the first application of SNNs to gait recognition, using traditional RGB camera-based data and a 4-layer CNN-based architecture. Although their model included 4.3 million parameters, it still demonstrated the typical advantages of SNNs, such as minimal overhead, low energy consumption, and improved efficiency.

III. Processing Pipeline

This work aims to accurately classify gait patterns using 3D skeleton data, foot pressure data, and a combination of both. Skeleton data consist of time sequences of 3D coordinates from 33 joints. For this reason, we employed a GRU-based encoder in our SkGRU model to extract meaningful features. In general, Recurrent Neural Networks (RNNs) are particularly well suited for this type of data and have demonstrated strong performance in skeleton-based abnormal gait classification, as confirmed by several studies ([2]- [5]). However, simple RNNs are prone to the vanishing gradient problem, which limits their ability to capture long-term dependencies. To address this issue, more advanced architectures such as Long Short-Term Memory (LSTM) and GRU were developed, both incorporating gating mechanisms. Considering the trade-off between efficiency and complexity, we opted for GRUs, which rely on an update gate and a reset gate. This results in a simpler structure compared to LSTMs, which have been proposed in other studies, while maintaining comparable performance and enabling more efficient computation. As a result, the SkGRU encoder is composed of four stacked GRU layers.

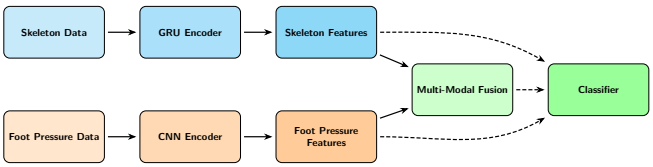


Fig. 1: Overview of the processing pipeline.

Foot pressure data can be interpreted as 1-channel images. Given the well-established performance of Convolutional Neural Networks in image processing and classification tasks, we opted for a CNN-based encoder to extract relevant features from this modality in our PrCNN model. This encoder is designed to progressively extract higher-level features from the sparse foot pressure images. It includes an initial series of convolutional and pooling layers, followed by a specialized block that emphasizes the most informative regions of the input.

For both SkGRU and PrCNN, we employed a shared classifier to evaluate the classification accuracy of the extracted features. But, since the ultimate goal is to combine both modalities, we also propose a hybrid classification model that merges the extracted features through a multi-modal fusion (as in [2]), performing gait classification using both data sources.

The entire pipeline described above is schematically illustrated in Fig. 1.

Finally, we implemented SkSNN, a skeleton-based single-modal Spiking Neural Network. This approach enables a simpler architecture by leveraging the inherently sparse and event-driven nature of spiking computation. As the SkSNN model lies outside the main processing pipeline, its design and implementation are discussed in detail in V-A5.

IV. Signals, Images and Features

Skeleton and foot pressure data were simultaneously collected in a controlled laboratory setup, shown in Fig. 2. Twelve healthy male subjects, familiar with the system and trained to simulate five pathological gaits (antalgic, lurching, steppage, stiff-legged, and Trendelenburg), walked along a 4-meter walkway while data acquisition was conducted under supervision. The collected dataset includes a total of 1,440 skeleton samples and 1,440 foot pressure samples, as each of the twelve subjects performed 20 trials for each of the six gait types ($12 \times 6 \times 20$).

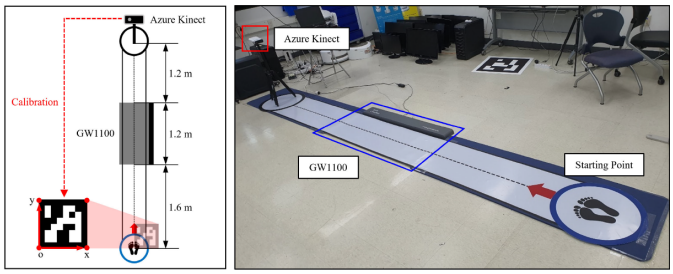


Fig. 2: Specifications and setup of the data collection system [2].

Skeleton information is represented as sequential datasets of 3D coordinates from 33 joints, collected using a single depth camera (Azure Kinect, Microsoft, USA). For each subject, one normal and five pathological gait types were recorded. The pathological gaits were simulated by limiting specific physical functions and following some guidelines [3].
Foot pressure data are collected from a 1280 mm $\times$ 480 mm sized pressure plate (GW1100, GHiWell, Korea) that can measure a maximum pressure of $100N/cm^2$. The foot pressure is the average of the plantar foot pressures in all time sequences. These data can be expressed as 1-channel images with input shape (1, 128, 48).
Fig. 3 shows examples of the samples analyzed in our study.

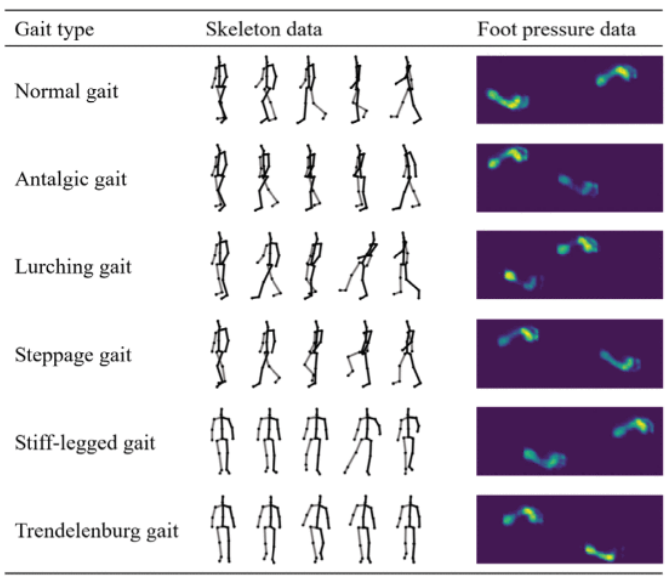


Fig. 3: Examples of skeleton and foot pressure data for the six gait types [2].

A. Normalization

We experimented with various normalization techniques for the skeleton data. The most effective method, which we adopted throughout our study, was inspired by [8] but not identical. It consists of centering the coordinates of each skeleton joint relative to the pelvis joint. Specifically, at each time step in the skeleton sequence, every coordinate (x, y, z) is translated so that the pelvis is always positioned at $(0, 0, 0)$. If the pelvis has coordinates (x_p, y_p, z_p), then each joint (x, y, z) is transformed into (x', y', z') as follows:

$$x' = x - x_p$$
$$y' = y - y_p$$
$$z' = z - z_p$$

This transformation aligns all skeletons relative to their pelvis joint, removing variations in global position, ensuring that motion analyses focus solely on the relative positions of joints and ensuring frame-to-frame comparability. Then, to compensate for differences in body size or proportions across subjects, we applied scale normalization by standardizing the distance between the left and right hip joints ($scale$):

$$x'' = x'/scale$$
$$y'' = y'/scale$$
$$z'' = z'/scale$$

Foot pressure data are normalized by dividing each value by 100 (the maximum measurable pressure in N/cm^2 of the pressure plate):

$$I_{\text{normalized}} = I_{\text{raw}}/100$$

Where:

- $I_{\text{normalized}}$ is the normalized image
- I_{raw} is the original image

B. Data Augmentation

1) Flipping: We augmented the training set in each fold by reversing the left and right sides of the skeleton and foot pressure data.
For foot pressure data, this augmentation consists simply of flipping the image horizontally. For skeleton data, it involves swapping the coordinates of the left and right joints without altering the XYZ reference system. For example, the coordinates of the right knee joint are assigned to the left knee joint and vice versa, effectively mirroring the skeleton structure while preserving spatial consistency.
Flipping effectively doubles the size of the training sets, introducing spatial variability (crucial since the task is invariant to left-right symmetry) in training data and improving generalization.

2) Random crop: The skeleton time-series do not have uniform lengths, ranging from a minimum of 118 to a maximum of 509 time steps. To address this variability, we applied random cropping to the training skeleton sequences. Specifically, for each training sequence, we randomly selected a valid starting point to extract a contiguous subsequence of a fixed crop length (100). This cropped portion is then used as input for the model. Random cropping increases the diversity of training data and improves model robustness by encouraging it to learn from different parts of the input. On the other hand, for validation data, we applied a center crop by extracting the 100 inner frames from each sequence.

3) Random rotation: Foot pressure data were also rotated by a fixed angle of 30° to introduce additional variability during training and to promote model robustness.

C. Train-validation split

SkGRU, PrCNN, and SkSNN are trained using a leave-four-subjects-out cross-validation (L4SO-CV). In each fold, 480 samples (corresponding to 4 subjects) are held out as the validation set, in order to ensure more consistent and reliable performance evaluation. This choice is motivated by the fact that, in these three cases, the training data is augmented: for SkGRU and SkSNN, the training set consists of $960 \times 2 = 1920$ samples, while for PrCNN, where foot pressure images are triplicated, the training set includes $960 \times 3 = 2880$ samples.

Instead, the hybrid model, which relies on the features extracted by the single-modal encoders without applying data augmentation, is trained using leave-one-subject-out cross-validation (LOSO-CV), holding out the 120 samples of one subject for validation in each fold.

V. Learning Framework

A. Models

To implement SkGRU, PrCNN and HyM we used TensorFlow, whereas SkSNN was developed using snnTorch.

1) Skeleton-based single-modal GRU (SkGRU) encoder: Our skeleton-based encoder consists of 4 GRU layers and a final fully connected layer as shown in Fig. 4(d). The input shape varies depending on the number of joints considered: it is (100, 27) when only leg joints (pelvis, hips, knees, ankles and feet) are included, and (100, 42) when spine_naval, spine_chest, neck and clavicles are added, totaling 14 joints. From the beginning, we excluded facial and arm joints, as they have been shown to negatively impact classification performance [3].

Gated Recurrent Units are recurrent neural architectures which capture long-term dependencies while mitigating the vanishing gradient problem. Two main gates control the information flow, managing past information to retain or discard:

- Update gate: Controls how much of the past information to retain
- Reset gate: Controls how much of the past information to forget

The GRU equations are the following.

$$
\begin{aligned}
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) \\
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) \\
\tilde{h}_t &= \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t
\end{aligned}
$$

Where:

- x_t is the input at time step t
- h_{t-1} is the previous hidden state
- h_t is the current hidden state
- σ is the sigmoid activation function
- $\odot$ denotes element-wise multiplication
- W_*, U_*, b_* are the learnable parameters

In our network, the first three GRU layers return the full sequence of hidden states, which are used as input to the subsequent GRU layers. The final GRU layer returns only the last hidden state, which is then passed to the dense layer for converting the features to the desired size. The resulting feature vector is then fed into the classifier described in Sec. V-A3.

2) Pressure-based single-modal CNN (PrCNN) encoder: To extract relevant features from foot pressure data, we implemented a CNN-based encoder. As shown in Figure 4(c), the input images have a shape of (1, 128, 48). The input is first processed through three blocks, referred to as Feature Blocks (see Figure 4(a)). Each Feature Block includes two 2D convolutional layers with 3×3 filters, stride of 1, ReLU activation, and zero-padding to preserve the input shape. A 2×2 max-pooling layer follows to reduce the spatial dimensions. The number of channels increases across blocks (16, 32, and 64, respectively) to progressively capture higher-level features. These operations play a crucial role in feature extraction and representation learning.

Subsequently, a block referred to as the Focus Block (Fig. 4(b)) is applied. It consists of a convolutional layer with 128 filters and the same settings as in the Feature Blocks, followed by a batch normalization layer and a pointwise convolution with ReLU activation. Since the signal present in foot pressure images is sparse, this design allows the network to emphasize and refine activations in the most informative spatial regions.

The feature maps are then flattened and passed through a fully connected layer with 128 neurons, followed by a dropout layer (see Sec. V-B) that drops 50% of the input units. Finally, a dense layer maps the output to the desired feature size. The resulting vector is then fed into the classifier (Sec. V-A3).

3) Classifier: The classifier, displayed in Fig. 4(e), is shared between SkGRU, PrCNN and the hybrid model (Sec. V-A4). The input features are first processed by a dropout layer (see Sec. V-B) with a drop rate of 0.5, followed by a batch normalization layer to regularize, stabilize, and accelerate the learning process. The core of the classifier consists of four fully connected layers with progressively decreasing dimensionality: 128, 64, 32, and finally 6 units, corresponding to the number of target classes. The first three dense layers include L2 regularization (Sec. V-B). The last layer is activated with a Softmax function, resulting in output class probabilities for multi-class classification.

4) Hybrid model (HyM): As Jun et al. did in [2], we propose a hybrid model using multi-modal fusion. SkGRU and PrCNN encoders are used to extract features from the skeleton and foot pressure data, respectively, and then we apply a feature-level data fusion (as shown in Fig. 1). In the hybrid model, the features are integrated to capture both the spatial-temporal dynamics of the body motion and the

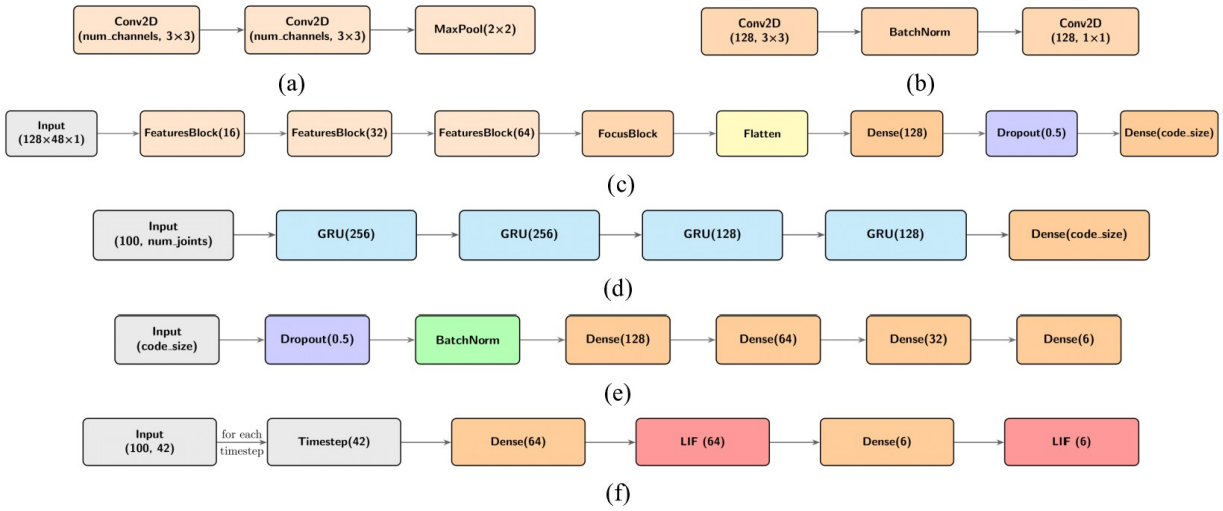


Fig. 4: Overview of the components and neural networks used: (a) Features Block, (b) Focus Block, (c) PrCNN encoder, (d) SkGRU encoder, (e) classifier model, (f) SkSNN.

plantar pressure distribution, allowing a more comprehensive representation for gait classification.

The extracted features $\mathbf{f}_s = (f_s^{(1)}, \ldots, f_s^{(N_s)})$ and $\mathbf{f}_p = (f_p^{(1)}, \ldots, f_p^{(N_p)})$, where N_s and N_p denote the numbers of extracted features from the skeleton-based and pressure-based encoding layers, respectively, were concatenated into the multi-modal features $\mathbf{f}_{s+p}$ as:

$$\mathbf{f}_{s+p} = (f_s^{(1)}, \ldots, f_s^{(N_s)}, f_p^{(1)}, \ldots, f_p^{(N_p)}).$$

5) Skeleton-based single-modal Spiking Neural Network (SkSNN): After analyzing which modality fitted best in our research scope, we implemented a Spiking Neural Network to model skeleton data, leveraging its event-driven processing to capture temporal patterns with high precision and efficiency.

The model, pictured in Fig. 4(f), is structured as a loop that iteratively scans the time steps of the skeleton time series, comprising two main layers that process sequentially each time step. The 42-dimensional input is first projected through a fully connected layer with 64 output units, which are gathered across time producing a tensor of shape (100, 64). A feature vector is extracted by summing this tensor along the time step dimension (first axis). This is followed by a Leaky Integrate-and-Fire (LIF) layer, whose resulting spikes are passed through a final dense layer and another LIF layer to generate the output spikes. These are accumulated into a (100, 6) tensor used for classification (see Sec. V-B).

Leaky Integrate-and-Fire neurons, modeled considering both biological plausibility and practicality, serve as a fundamental computational unit of the SNN. A LIF neuron takes the sum of the weighted inputs like an artificial neuron but, instead of passing it directly to an activation function, it integrates the input over time with a leakage, much like an RC circuit. If the integrated value exceeds a threshold, then the LIF neuron will emit a voltage spike. LIF dynamics in discrete time:

$$U[n] = \beta U[n-1] + WX[n-1] - \theta S[n-1]$$

Where:

- $U[n]$ is the membrane potential at time n
- β is the membrane potential decay rate
- $WX[n]$ is the weighted input at time n
- θ is the threshold for spike generation
- $S[n]$ are the spikes generated at time t.

In this formulation, $U[n]$ has the threshold subtracted whenever the neuron emits a spike. However, in our SkSNN, we employed `reset_mechanism='zero'`, which resets $U[n]$ to zero upon spike emission.

B. Training

All models were trained using the ADAM optimizer with a learning rate of 0.0001, except for the SkSNN, which used a learning rate of 0.001.

As loss function, we used sparse categorical cross-entropy for SkGRU, PrCNN, and HyM, while cross-entropy spike count loss was used for SkSNN. The latter sums up the spikes across the time steps and applies a cross-entropy loss, where the predicted class is the one with the highest total spike count. This approach encourages the correct class to fire consistently across all time steps while suppressing spike activity from incorrect classes. Furthermore, for the SkSNN model, the membrane potential decay rate β and the threshold θ were included among the learnable parameters. The surrogate gradient function used was the Fast Sigmoid.

Regularization: In each of our models we applied Early Stopping and Model Checkpoint techniques, and in the shared classifier (Sec. V-A3) we employed Dropout and L2 regularization.

Early Stopping prevents overfitting and avoids an intense use of computational resources; we set 10 as patience parameter,

halting training if the validation loss did not improve for 10 consecutive epochs. Model Checkpoint technique saves the model at the epoch that achieved the lowest validation loss.
Dropout was applied with a drop rate of 0.5, meaning that each neuron in the layer had a 50% chance of being deactivated during each training step. This reduces reliance on specific neurons and encourages the network to learn more robust representations. L2 regularization with a penalty coefficient of 0.01 was used to control the magnitude of the weights, helping to prevent overfitting and improve generalization.

VI. Results

The experiments have been conducted on the free Google Colab environment using T4 GPU.

A. Models performance

To determine which feature dimension yielded the best performance, we conducted a grid search across different sizes. The results are reported in Tab. 1. The code sizes that achieved the best performance were 64 for the SkGRU and 16 for the PrCNN.

Model	code_size	only_legs	Accuracy	Precision	Recall	F1 score
SkGRU	16	False	0.936	0.938	0.936	0.935
	16	True	0.862	0.867	0.862	0.854
	32	False	0.940	0.941	0.940	0.939
	32	True	0.871	0.869	0.871	0.868
	64	False	0.942	0.943	0.942	0.941
	64	True	0.862	0.859	0.862	0.860
PrCNN	8	-	0.464	0.463	0.464	0.448
	16	-	0.489	0.505	0.489	0.473
	32	-	0.479	0.478	0.479	0.465
HyM	80	-	**0.948**	**0.949**	**0.948**	**0.948**
SkSNN	64	False	0.832	0.846	0.832	0.833

TABLE 1: Average performance metrics of our models.

Furthermore, for the SkGRU, we evaluated two different joint configurations, as described in Sec. V-A1, controlled by the parameter `only_legs`. When `only_legs=False`, 14 joints were used, while setting it to `True` limited the input to 9 lower-limb joints. Across all tested feature dimensions for skeleton data, the configuration using 14 joints consistently yielded better results, as it can be seen in Tab. 1.
After identifying the best-performing configurations, we combined the extracted features to create a multi-modal input of size $64 + 16 = 80$ for the hybrid model.
The average performance of HyM, shown in the last row of Tab. 1, slightly surpasses that of the best SkGRU model (0.948 vs. 0.942 accuracy). In contrast, the PrCNN models perform considerably worse, with the best configuration reaching only 48.3% accuracy, substantially lower than the 68.82% achieved by DenseNet201 in [2]. Nevertheless, it is worth considering that our model is significantly lighter (20 million parameters vs 1.7 million).
Given that the improvement achieved through feature-level multi-modal fusion is minimal, and considering, as further discussed in Sec. VI-E, that using only skeleton data results in significantly faster inference and fewer parameters, we decided to implement only the skeleton-based single-modal SNN, foregoing the foot pressure and multi-modal fusion of features. We used the best-performing parameters of the SkGRU also for the SkSNN. Although our SkSNN is a relatively simple model, it achieves an overall accuracy of 0.832 (last row of Tab. 1), lower than SkGRU and HyM, but significantly outperforming PrCNN.

B. Training and Validation curves

Fig. 5 displays the training and validation accuracy and loss curves for the best-performing SkGRU, PrCNN and HyM models.

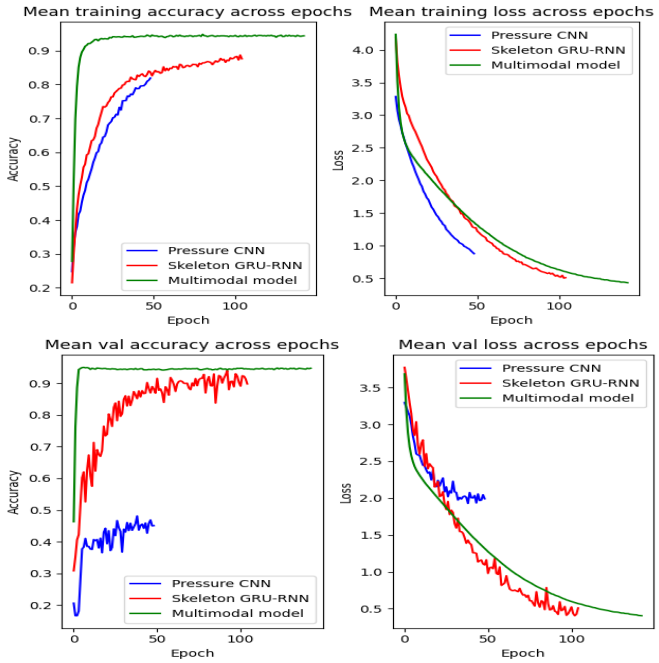


Fig. 5: Accuracy and loss curves of SkGRU, PrCNN and HyM.

For what concerns accuracy, the hybrid model achieves the best performance, requiring only a few epochs to reach high values in both training and validation. In contrast, SkGRU takes longer to attain comparable results, while PrCNN consistently demonstrates lower accuracy across all training phases. Regarding the loss curves, HyM shows the fastest convergence during the initial training stages, while SkGRU achieves better performance after approximately 40 epochs. This pattern is also evident in the validation loss curves. In contrast, the PrCNN training loss decreases rapidly but performs significantly worse on the validation set, indicating clear signs of overfitting.

Fig.6 shows the accuracy and loss curves of the SkSNN, which are presented separately from the other three models due to the use of a different loss function, as described in Sec.V-B. This model requires the fewest epochs to train and shows a steady improvement trend. Similarly to PrCNN and SkGRU, the validation accuracy and loss exhibit fluctuations compared to the training curves.

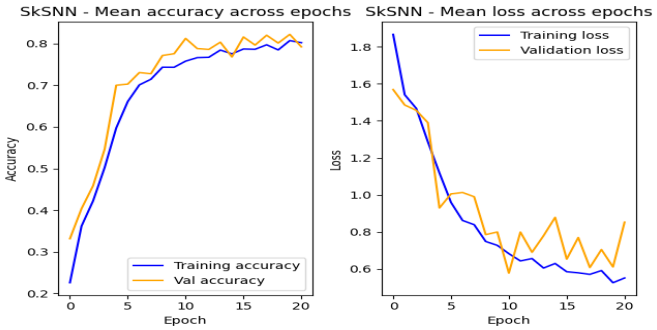


Fig. 6: Accuracy and loss curves of the SkSNN.

C. t-SNE

We applied t-distributed Stochastic Neighbor Embedding (t-SNE) to the extracted features of each well-trained model to observe their ability in separating the classes. The t-SNE is a non-linear dimensionality reduction technique, well suited to visualize high-dimensional data in 2 or 3 dimensions. t-SNE tries to preserve the local structure, focusing on neighborhoods, of data by keeping similar points in the high-dimensional space close together in the low-dimensional space.

This analysis confirmed the classification capability of the chosen models. In Figure 7, we report an example of how SkGRU, PrCNN, HyM and SkSNN can discriminate between different gaits considering a single subject. SkGRU and HyM exhibit similar performance, although HyM performs slightly better for some subjects, as observed in this case in which emerges a better discrimination between antalgic and Trendelenburg gait. In contrast, PrCNN shows comparatively poor performance: the only gait it seems to separate is the normal one. Regarding SkSNN, the model demonstrates significantly better discriminative capability than PrCNN, though it performs slightly worse than SkGRU and HyM. Nevertheless, its performance appears promising and suggests potential for future improvements and, eventually, a multi-modal hybrid SNN.

D. Gait-specific performance

Observing Fig. 8 we can analyze gait-specific performance across the models. The average performance of PrCNN, as mentioned above, is the worst. Concerning single gait types, it is evident that this architecture encounters significant difficulties in classifying lurching and Trendelenburg gaits, which are mostly mistaken for the stiff-legged and the steppage (Fig. 8(b)). The performance of SkGRU and HyM is very similar, especially with respect to the stiff-legged, lurching, steppage, and Trendelenburg gaits (Fig. 8(a) and 8(c)). It is interesting to notice that SkGRU achieves better results in normal gait classification (233 vs 217 correctly classified instances), whereas HyM outperforms SkGRU in the antalgic gait classification (220 vs 199 correctly classified instances). In both models, as well as in [2], the antalgic gait and the Trendelenburg gait are sometimes misclassified as each other. Furthermore, the skeleton-based single-modal GRU model tends to misclassify antalgic gait patterns as normal, with 26 such instances, while the hybrid model reduces this confusion to only 12 cases. Conversely, HyM shows a higher tendency to misclassify normal gait as antalgic, with 20 misclassifications compared to just 7 in SkGRU.

SkSNN demonstrates strong overall performance with an accuracy of 0.832. However, as shown in Fig. 8(d), the accuracy drops for certain classes. Notably, normal and antalgic gaits are sometimes misclassified as each other, with the model particularly struggling to identify antalgic gait (only 166 instances are correctly classified). Additionally, steppage and Trendelenburg gaits are frequently misclassified as normal, with 19 and 28 such cases, respectively. The performance on Trendelenburg gait is also below average, with only 169 correctly classified instances.

In summary, SkGRU performs better in classifying normal gait patterns, while HyM demonstrates superior ability in distinguishing between different types of pathological gaits. The worse overall performance of SkSNN is explained by modest results within the antalgic and Trendelenburg classes.

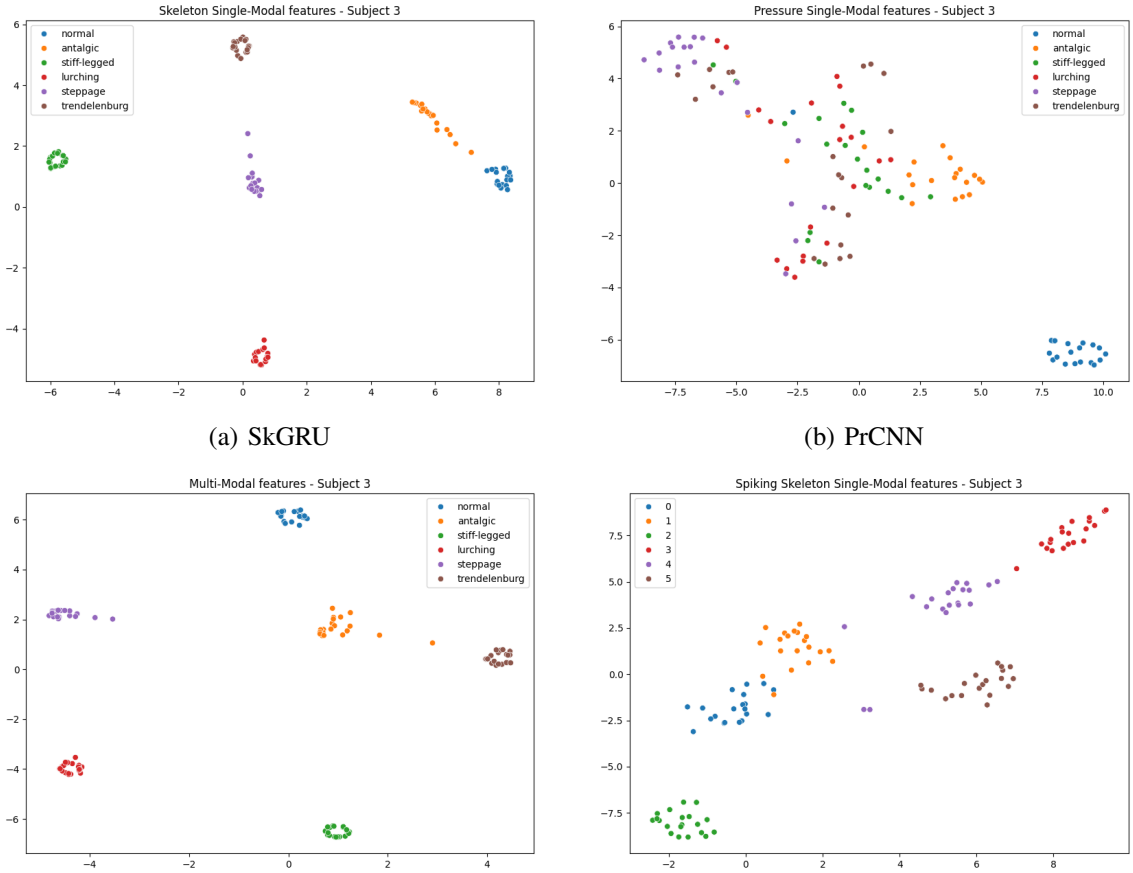
E. Efficiency-aware model comparison

Tab. 2 shows, for each model, the achieved accuracy, the number of parameters, and the mean inference time. Through these key metrics, it is possible to have an idea of spatial cost, performance, and temporal cost. Although HyM achieves the highest accuracy, it is the most expensive model in terms of both space (2.63 million parameters) and mean inference time (6.27 ms). Given the negligible difference of accuracy between HyM and SkGRU (0.948 vs 0.942), the latter model is a better choice because it allows saving spatial and temporal resources thanks to considerably lower number of parameters (895k, which is only 34% of HyM's size) and mean inference time (3.67 ms). SkSNN could be the most suitable choice due to its very low complexity (314k parameters, amounting to only 12% of HyM's total) while still ensuring acceptable performance. Regarding mean inference time, it is slightly higher than SkGRU but considerably lower with respect to HyM. Especially when the trade-off leans more towards spatial complexity, SkSNN becomes a valuable option. Furthermore, the model adopts a minimal architecture, with only two layers and 100 time-step iterations. This simplicity offers significant potential for future enhancements, particularly in terms of inference efficiency and input processing, potentially improving performance and reducing temporal costs without compromising the lightweight nature of the model.

Since it has a very poor performance, it is not worth considering PrCNN.

Model	Parameters	Accuracy	Mean Inference Time (ms)
SkGRU	895,062	0.942	**3.6748**
PrCNN	1,750,502	0.489	4.4573
HyM	2,639,126	**0.948**	6.2696
SkSNN	**314,200**	0.832	4.3892

TABLE 2: Model comparison with total parameters, accuracy, and mean inference time.



(a) SkGRU

(b) PrCNN

(c) HyM

(d) SkSNN

Fig. 7: t-SNE visualization for subject 3 features extracted using: (a) SkGRU, (b) PrCNN, (c) HyM, and (d) SkSNN.

VII. Concluding Remarks

A. Summary

The main idea of the study was to combine skeleton and foot pressure data to obtain a highly accurate gait classification. To do so, we modeled data through skeleton-based GRU encoders and foot pressure-based CNN encoders, and, after choosing the best architectures, we merged the extracted features through a multi-modal fusion to develop a hybrid classification model. Its performances were compared to the single-modal classifications which relied solely on either skeleton or pressure data. Consequently, we implemented a skeleton-based single-modal SNN in order to exploit the advantages of spiking-based models, especially in terms of computational resources economy.

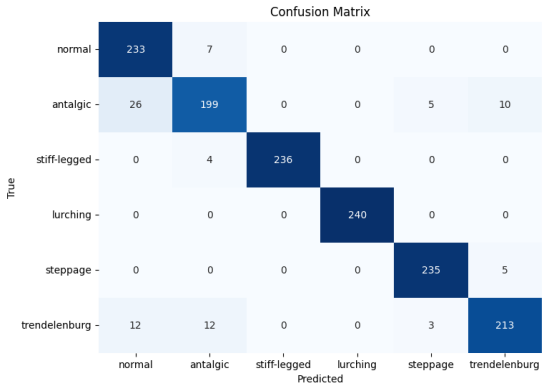
B. Difficulties

During the research, we attempted to implement also a pressure-based single-modal SNN but we faced several difficulties and the performance obtained was really poor. Even with a very basic convolutional SNN containing a minimal number of parameters, the model exhibited significant overfitting, with the validation loss often failing to converge. This behavior suggests that the network easily memorized patterns from the training data but struggled to generalize. This led us to question the quality of the dataset. The subjects mimicked the pathological gaits, which may have compromised the authenticity of the foot pressure recordings. While imitating gait dynamics might be relatively straightforward, doing the same with foot pressure patterns is likely much more challenging. This idea was supported by the sparse nature of foot pressure images, which initially led us to believe that SNNs, due to their event-based essence, could yield better performance. However, this was not the case.

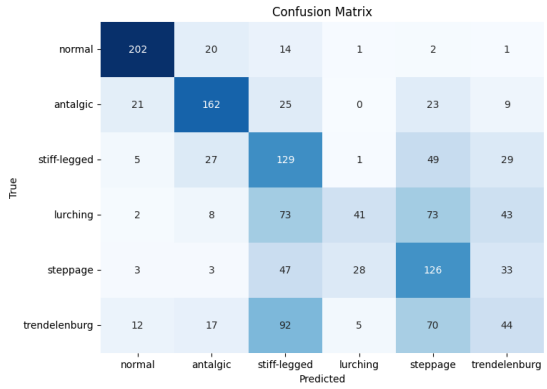
C. Future developments

The idea proposed by Zhou et al. [12] can be an interesting foundation for future developments. Their lightweight architecture is designed to monitor post-stroke patients' gait. Similarly, our models, which have a limited computational cost, could be applied, with suitable modifications, in a follow-up scenario even in a domestic monitoring setting.

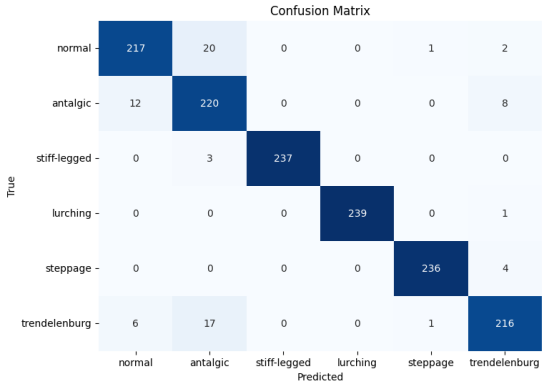
Despite the difficulties mentioned in Sec. VII-B, developing an SNN based on foot pressure data (employing datasets that involve the participation of individuals actually affected by gait disorders) remains a viable direction for future work. In



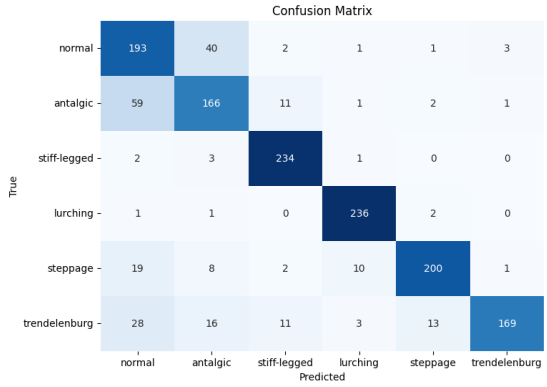
(a) SkGRU



(b) PrCNN



(c) HyM



(d) SkSNN

Fig. 8: Confusion matrices: SkGRU, PrCNN, HyM, and SkSNN.

particular, the use of alternative spiking architectures, such as SpikFormer [13], provides a promising foundation for extending gait analysis toward more advanced spiking-based approaches.

D. Lessons learned

First and foremost, the idea of developing models with potential application in clinical practice, which could save time and support healthcare professionals, was a strong source of motivation for us. From a technical standpoint, although challenging, exploring Spiking Neural Networks has given us a broader perspective on a novel and promising direction in the field of deep learning.

References

[1] "Gait definition." https://medical-dictionary.thefreedictionary.com/gaits.

[2] K. Jun, S. Lee, D.-W. Lee, and M. S. Kim, "Deep learning-based multimodal abnormal gait classification using a 3d skeleton and plantar foot pressure," *IEEE Access*, vol. 9, pp. 161576–161589, November 2021.

[3] K. Jun, Y. Lee, S. Lee, D.-W. Lee, and M. S. Kim, "Pathological gait classification using kinect v2 and gated recurrent neural networks," *IEEE Access*, vol. 8, pp. 139881–139891, July 2020.

[4] K. Jun, D.-W. Lee, K. Lee, S. Lee, and M. S. Kim, "Feature extraction using an rnn autoencoder for skeleton-based abnormal gait recognition," *IEEE Access*, vol. 8, pp. 19196–19207, January 2020.

[5] D.-W. Lee, K. Jun, S. Lee, J.-K. Ko, and M. S. Kim, "Abnormal gait recognition using 3d joint information of multiple kinects system and rnn-lstm," in *2019 41st Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, pp. 542–545, October 2019.

[6] M. Khokhlova, C. Migniot, A. Morozov, O. Sushkova, and A. Dipanda, "Normal and pathological gait classification lstm model," *Artificial Intelligence in Medicine*, vol. 94, pp. 54–66, March 2019.

[7] M. T. Naseem, H. Seo, N.-H. Kim, and C.-S. Lee, "Pathological gait classification using early and late fusion of foot pressure and skeleton data," *Applied Sciences*, vol. 14, January 2024.

[8] R. Baptista, G. Demisse, D. Aouada, and B. Ottersten, "Deformation-based abnormal motion detection using 3d skeletons," in *2018 Eighth International Conference on Image Processing Theory, Tools and Applications (IPTA)*, pp. 1–6, 2018.

[9] "Joints indeces." https://learn.microsoft.com/ko-kr/previous-versions/azure/kinect-dk/body-joints.

[10] J. K. Eshraghian, M. Ward, E. O. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu, "Training spiking neural networks using lessons from deep learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.

[11] Y. Tao, C.-H. Chang, S. SaÃ¯ghi, and S. Gao, "Gaitspike: Event-based gait recognition with spiking neural network," in *2024 IEEE 6th International Conference on AI Circuits and Systems (AICAS)*, pp. 357–361, 2024.

[12] C. Zhou, D. Feng, S. Chen, N. Ban, and J. Pan, "Portable vision-based gait assessment for post-stroke rehabilitation using an attention-based lightweight cnn," *Expert Systems with Applications*, vol. 238, p. 122074, March 2024.

[13] Z. Zhou, Y. Zhu, C. He, Y. Wang, S. Yan, Y. Tian, and L. Yuan, "Spikformer: When spiking neural network meets transformer," 2022.